COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L FULL STACK APPLICATION DEVELOPMENT

S. LALITH ADITYA 2400031810

#SKILL- 7 □ REST API - CRUD Operations using ResponseEntity

Prerequisites:

- General Idea on Spring Boot MVC Architecture
- Knowledge of Service and Controller layers

A university application needs backend services to manage course details such as courseId, title, duration, and fee. The admin should be able to add, update, delete, and view course information. The system must return proper HTTP status codes and structured responses for reliable communication between client and server.

Your task is to implement REST endpoints using ResponseEntity to perform complete CRUD operations on course data.

Tasks:

1. Create a Course entity with fields: courseId, title, duration, fee.

```java
package com.university.course.model;

import jakarta.persistence.Entity;

@Entity
public class Course {

    @Id
    private int courseId;
    private String title;
    private String duration;
    private double fee;

    public Course() {}

    public Course(int courseId, String title, String duration, double fee) {
        this.courseId = courseId;
        this.title = title;
        this.duration = duration;
        this.fee = fee;
    }

    public int getCourseId() {
        return courseId;
    }

    public void setCourseId(int courseId) {
        this.courseId = courseId;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getDuration() {
        return duration;
    }

    public void setDuration(String duration) {
        this.duration = duration;
    }
```

2. Create a Service layer that performs full CRUD operations.

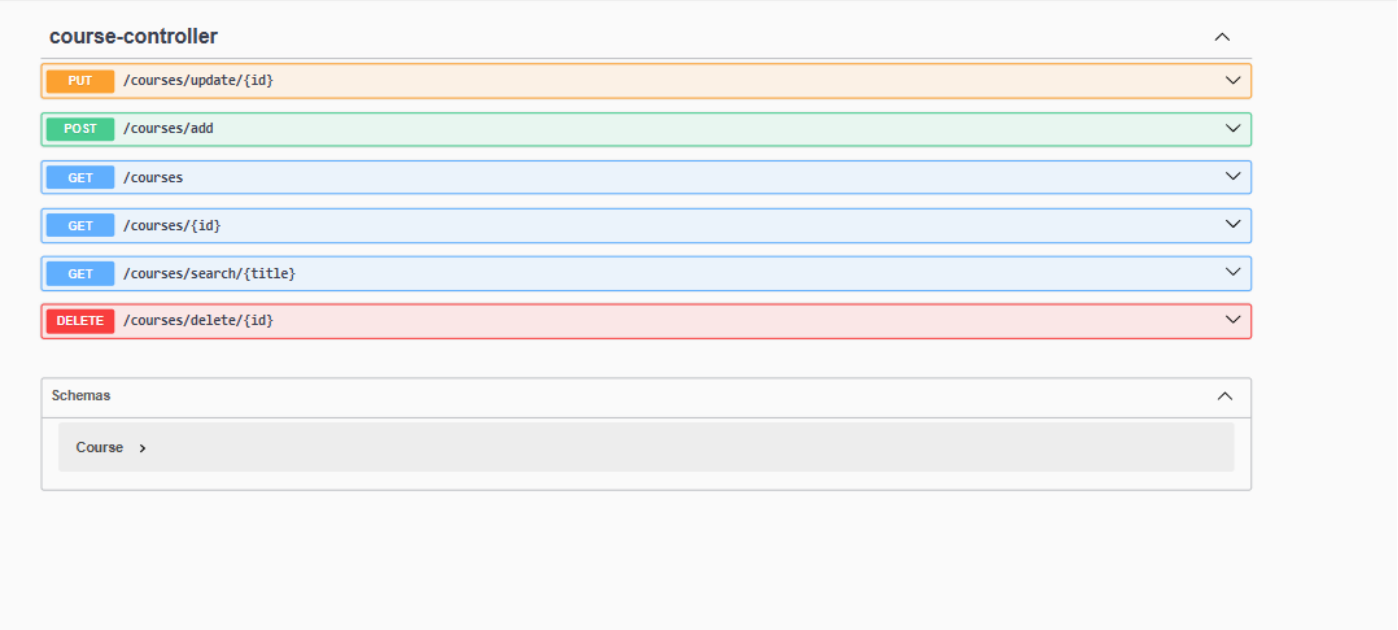
```java
package com.university.course.service;

import java.util.List;

@Service
public class CourseService {

    @Autowired
    private CourseRepository repo;

    public Course addCourse(Course course) {
        return repo.save(course);
    }

    public List<Course> getAllCourses() {
        return repo.findAll();
    }

    public Optional<Course> getCourseById(int id) {
        return repo.findById(id);
    }

    public Course updateCourse(int id, Course course) {
        course.setCourseId(id);
        return repo.save(course);
    }

    public void deleteCourse(int id) {
        repo.deleteById(id);
    }

    public List<Course> searchByTitle(String title) {
        return repo.findByTitleContainingIgnoreCase(title);
    }
}
```

3. Implement REST endpoints using @PostMapping, @PutMapping, @DeleteMapping, and @GetMapping.

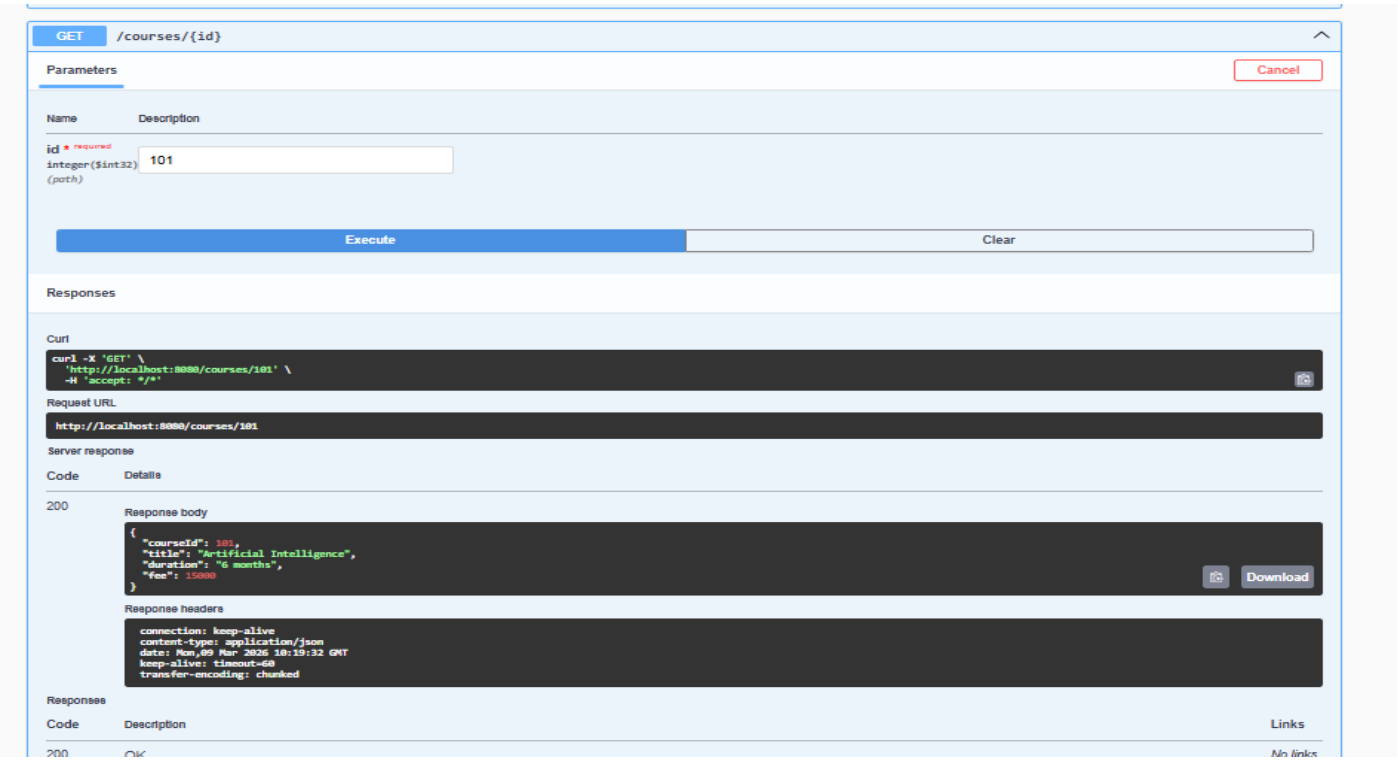
```java
package com.university.course.controller;

import java.util.List;

@RestController
@RequestMapping("/courses")
public class CourseController {

    @Autowired
    private CourseService service;

    @PostMapping("/add")
    public ResponseEntity<?> addCourse(@RequestBody Course course) {
        try {
            Course c = service.addCourse(course);
            return new ResponseEntity<>(c, HttpStatus.CREATED);
        } catch (Exception e) {
            return new ResponseEntity<>("Course creation failed", HttpStatus.BAD_REQUEST);
        }
    }

    @GetMapping
    public ResponseEntity<List<Course>> getAllCourses() {
        return new ResponseEntity<>(service.getAllCourses(), HttpStatus.OK);
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getCourse(@PathVariable int id) {
        Optional<Course> course = service.getCourseById(id);

        if(course.isPresent())
            return new ResponseEntity<>(course.get(), HttpStatus.OK);
        else
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
    }

    @PutMapping("/update/{id}")
    public ResponseEntity<?> updateCourse(@PathVariable int id, @RequestBody Course course) {

        Optional<Course> existing = service.getCourseById(id);

        if(existing.isPresent()) {
            return new ResponseEntity<>(service.updateCourse(id, course), HttpStatus.OK);
        } else {
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
```

```java
SchedulerEngine...  AppointmentRequ...  PriorityScore.ja...  Appointment.java  AppointmentProp...  Pa
    public ResponseEntity<?> getCourse(@PathVariable int id) {
        Optional<Course> course = service.getCourseById(id);

        if(course.isPresent())
            return new ResponseEntity<>(course.get(), HttpStatus.OK);
        else
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
    }

    @PutMapping("/update/{id}")
    public ResponseEntity<?> updateCourse(@PathVariable int id, @RequestBody Course course) {

        Optional<Course> existing = service.getCourseById(id);

        if(existing.isPresent()) {
            return new ResponseEntity<>(service.updateCourse(id, course), HttpStatus.OK);
        } else {
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
        }
    }

    @DeleteMapping("/delete/{id}")
    public ResponseEntity<?> deleteCourse(@PathVariable int id) {

        Optional<Course> existing = service.getCourseById(id);

        if(existing.isPresent()) {
            service.deleteCourse(id);
            return new ResponseEntity<>("Course deleted successfully", HttpStatus.OK);
        } else {
            return new ResponseEntity<>("Course not found", HttpStatus.NOT_FOUND);
        }
    }

    @GetMapping("/search/{title}")
    public ResponseEntity<?> searchCourse(@PathVariable String title) {

        List<Course> courses = service.searchByTitle(title);

        if(courses.isEmpty())
            return new ResponseEntity<>("No courses found", HttpStatus.NOT_FOUND);
        else
            return new ResponseEntity<>(courses, HttpStatus.OK);
    }
}
```

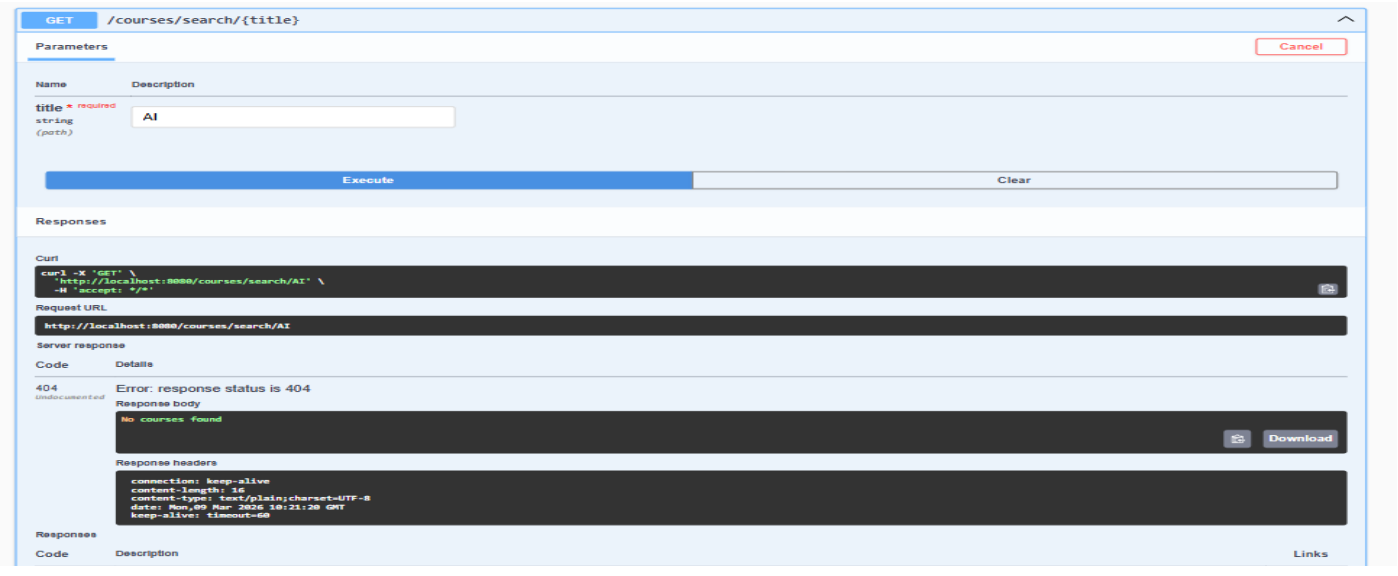
4. Use ResponseEntity to return:
   a. Success messages
   b. Error messages
   c. Status codes such as OK, CREATED, NOT_FOUND, BAD_REQUEST



5. Add a search endpoint /courses/search/{title} to filter courses by title.

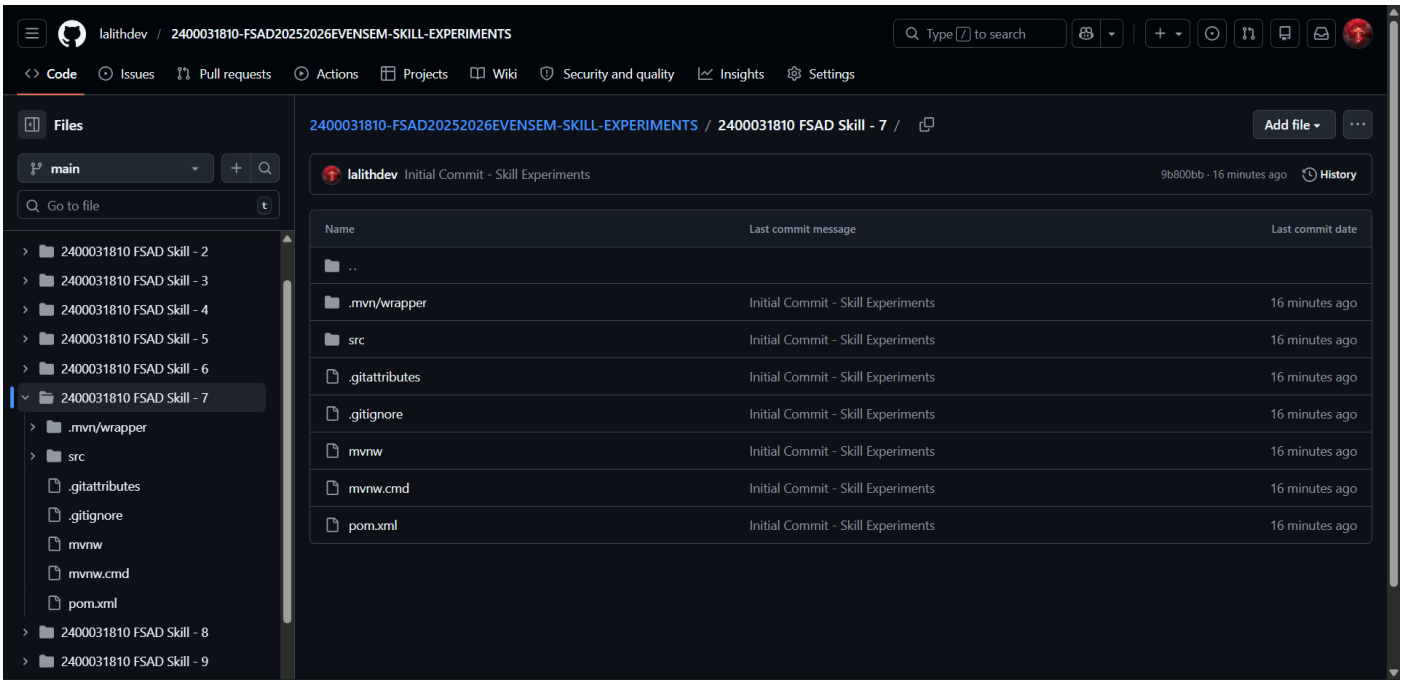


6. Test all endpoints (valid and invalid cases) using Postman.



7. Push the Spring Boot project to GitHub.

   https://github.com/lalithdev/2400031810-FSAD20252026EVENSEM-SKILL-EXPERIMENTS/tree/main/2400031810%20FSAD%20Skill%20-%207.git



VIVA QUESTIONS:

1. **What is ResponseEntity used for?**
   ResponseEntity is used to control the **entire HTTP response** in a REST API, including the **response body, HTTP status code, and headers**.
2. **What is the difference between @PostMapping and @PutMapping?**
   @PostMapping is used to **create new resources** in the server.
   @PutMapping is used to **update an existing resource**.
3. **Why are HTTP status codes important in REST APIs?**
   HTTP status codes inform the client about the **result of the request**, such as success (200 OK), resource created (201 CREATED), or error (404 NOT_FOUND).
4. **What happens if an invalid ID is passed to an endpoint?**
   The server typically returns a **404 NOT_FOUND status code** with an error message indicating that the resource does not exist.
5. **Can ResponseEntity return both data and status codes together?**
   Yes. ResponseEntity can return **response data (body) along with HTTP status codes and headers** in the same response.

(For Evaluator's use only)

Comment of the Evaluator (if Any)

Evaluator's Observation

Marks Secured:_____out of _____

EMP ID & Full Name of the Evaluator:

Signature of the Evaluator Date of Evaluation: